Просто DevOps

20 самых частых вопросов на собесе DevOps

Собрано	**2 206 вопросов с настоящих собесов**
В деле	**20 вопросов с ответами**
Составил	**Просто DevOps**



Как работать с досье

Здесь 20 вопросов, которые чаще всего задают на собеседовании DevOps-инженера. Мы отобрали их из 2 206 вопросов с настоящих собесов и коротко ответили на каждый.

1. Прочитай вопрос и ответь вслух, как на собесе. В ответ пока не подглядывай.
2. Сверься с ответом. Не совпало — перечитай и вернись к вопросу через пару дней.
3. Начни сверху: первые вопросы задают чаще всего.



Такая бирка висит у каждого вопроса: столько раз его спрашивали — так часто он встретился в записях собесов, которые мы собрали.



так и тренируйся

Вопрос № 01 тема: Kubernetes

Liveness и Readiness Probe — что они собой представляют, за что отвечают?



Обе пробы — это kubelet регулярно проверяет контейнер: HTTP-запросом, TCP-подключением, командой внутри или через gRPC. Liveness спрашивает «ты жив?»: несколько провалов подряд — и kubelet перезапускает контейнер. Readiness спрашивает «готов принимать трафик?»: пока нет, под убирают из эндпоинтов Service, но не перезапускают. Если приложение долго стартует, добавь `startupProbe`, чтобы liveness не убила его на старте.

Вопрос № 02 тема: Linux

Для чего нужен Load Average и что показывают три числа?



Это очередь к процессору и диску: сколько процессов работает, ждёт процессор или ждёт диск. Три числа — среднее за 1, 5 и 15 минут. Смотреть их надо рядом с числом ядер: на 4 ядрах load 4 — загружено полностью, 8 — процессы уже стоят в очереди. Первое число больше третьего — нагрузка растёт, меньше — спадает.

Вопрос № 03 тема: Kubernetes

В чём разница между requests и limits? Будет ли Kubernetes лимитировать поды, если лимиты не указаны явно?



`requests` — сколько ресурсов контейнер просит заранее: по ним планировщик ищет ноду, где столько свободно. `limits` — потолок. Упрёшься в лимит CPU — контейнер начнут притормаживать (throttling), в лимит памяти — его убьёт OOM killer. Лимитов не указал — контейнер может съесть всю свободную ноду. Сам Kubernetes его не ограничит, если только в namespace нет `LimitRange` с лимитами по умолчанию.

Вопрос № 04 тема: Docker

Какие best practice по написанию Dockerfile и оптимизации образов?



Начни с маленького официального образа с точной версией, например `python:3.12-slim`. Сборку раздели на стадии: компилятор и исходники остаются в первой, в итоговый образ едет только результат. Файлы зависимостей копируй раньше кода — тогда кэш не слетает при каждой правке. Временные файлы удаляй в том же `RUN`, лишнее отсекай через `.dockerignore`, запускай процесс не от root, а секреты передавай через `--mount=type=secret`.

Вопрос № 05 тема: Ansible

Почему важно знать variable precedence?



В Ansible одну и ту же переменную можно задать в куче мест: в defaults роли, в инвентаре, в плее, в vars роли, в командной строке. Какое значение победит, решает порядок приоритета, и уровней в нём 22. В самом низу значения командной строки вроде `-u`, сразу над ними `defaults` роли, на самом верху `-e` (extra vars). Не знаешь порядок — будешь долго гадать, откуда на сервере взялось не то значение.

Вопрос № 06 тема: Kubernetes

Из каких основных компонентов состоит Kubernetes и за что они отвечают?



Кластер — это «мозг» (control plane) и рабочие ноды. В мозге: `kube-apiserver` принимает все запросы, `etcd` хранит состояние кластера, `kube-scheduler` решает, на какую ноду поставить под, `kube-controller-manager` следит, чтобы в кластере было ровно то, что описано. На каждой ноде: `kubelet` запускает поды через containerd, а `kube-proxy` настраивает сетевые правила для Service.

Вопрос № 07 тема: Kubernetes

Чем Deployment, StatefulSet и DaemonSet отличаются друг от друга?



Deployment — для одинаковых подов без состояния: любой можно заменить другим, обновляются они постепенно. StatefulSet — когда у каждого пода должна быть своя личность: постоянное имя (`db-0` , `db-1`), свой диск и запуск по порядку. Так поднимают базы и кластеры. DaemonSet ставит ровно по одному поду на каждую подходящую ноду — так разворачивают агентов логов и мониторинга.

Вопрос № 08 тема: Docker

Чем CMD отличается от ENTRYPOINT?



`ENTRYPOINT` — какую программу запустит контейнер. `CMD` — аргументы к ней по умолчанию, а если `ENTRYPOINT` нет, то вся команда целиком. Всё, что напишешь после имени образа в `docker run` , заменит `CMD` . А `ENTRYPOINT` так просто не перебить — только флагом `--entrypoint` .

Вопрос № 09 тема: Docker

Какие механизмы ядра изолируют контейнер?



Две главные вещи. Пространства имён (namespaces) решают, что процесс видит: свои процессы, сеть, точки монтирования, имя хоста, пользователей. Контрольные группы (cgroups) решают, сколько он может взять: процессор, память, число процессов, ввод-вывод. Сверху права урезают через capabilities, seccomp и AppArmor или SELinux. И помни: ядро у контейнеров общее с хостом.

Вопрос № 10 тема: Docker

Для чего нужна multi-stage сборка?



Чтобы в итоговый образ не попал весь строительный мусор. В Dockerfile несколько `FROM`, каждый начинает свою стадию. В первой есть компилятор и всё для сборки, а в последнюю через `COPY --from` переносится только готовая программа. Образ выходит меньше, исходников и инструментов в нём нет — значит, и уязвимостей меньше.

Вопрос № 11 тема: Linux

Почему df и du могут сильно различаться?



Они считают по-разному. `df` берёт занятое место из счётчиков файловой системы, а `du` обходит файлы по именам и складывает их размеры. Чаще всего разница — это удалённый файл, который всё ещё держит открытым какой-то процесс: имени уже нет, `du` его не видит, а место занято. Найти такие файлы поможет `lsof +L1`.

Вопрос № 12 тема: CI/CD

Как бы ты построил пайплайн для нового сервиса?



Начал бы с быстрых проверок — линтер и unit-тесты, чтобы падать как можно раньше. Потом один раз собрал бы артефакт, например образ с тегом по коммиту, и дальше гонял бы именно его: интеграционные тесты, сканер уязвимостей. Потом публикация в реестр, деплой на stage, smoke-тест и выкатка на прод по кнопке. Для прода заранее подумал бы про права, секреты и откат на прошлую версию.



Вопрос № 13 тема: Базы данных

База работает медленно, пользователи жалуются. С чего начнёте разбор?

Сначала понял бы, что именно тормозит и с какого момента: всё или один сценарий, после релиза или когда выросла нагрузка. Потом пошёл бы по слоям: ресурсы сервера, соединения и блокировки в `pg_stat_activity`, самые тяжёлые запросы в `pg_stat_statements` и их планы через `EXPLAIN ANALYZE`. Индекс, переписанный запрос или пулер — только после замеров, а не наугад.

Вопрос № 14 тема: Kubernetes



Какие типы Service бывают в Kubernetes?

`ClusterIP` — внутренний адрес, виден только внутри кластера, это тип по умолчанию. `NodePort` вдобавок открывает один и тот же порт на каждой ноде. `LoadBalancer` вдобавок просит облако создать внешний балансировщик. `ExternalName` стоит особняком: отдаёт DNS-запись CNAME на внешнее имя и трафик не проксирует. И есть headless Service (`clusterIP: None`): DNS отдаёт адреса подов напрямую.

Вопрос № 15 тема: Kubernetes



В чём разница между Ingress и Ingress Controller?

Ingress — это просто правила: какой хост и путь в какой Service отправить и какой TLS-сертификат взять. Сам по себе он трафик не принимает. Ingress Controller — программа в кластере (ingress-nginx, Traefik, HAProxy), которая читает эти правила, собирает из них настройки прокси и уже принимает запросы. Нет контроллера — Ingress просто лежит и ничего не делает.

Вопрос № 16 тема: Docker

Чем контейнеризация отличается от виртуализации? Когда что используется?



Виртуальная машина — отдельный компьютер со своей ОС и своим ядром поверх гипервизора. Контейнер — обычный процесс на хосте, который изолировали через namespaces и ограничили через cgroups, ядро у всех общее. Поэтому контейнер стартует за доли секунды и весит мегабайты, а ВМ изолирована надёжнее. ВМ берут, когда нужна другая ОС, своё ядро или жёсткая граница между клиентами, контейнеры — для приложений, CI и микросервисов.

Вопрос № 17 тема: Linux

Почему может появиться No space left on device, хотя df показывает свободное место?



Чаще всего кончились не байты, а inode — проверь `df -i`. Ещё варианты: в этот путь смонтирован отдельный раздел, и забит именно он; место держит удалённый файл, который не отпускает процесс (`lsof +L1`); сработала квота пользователя; или остались только блоки, зарезервированные для root.

Вопрос № 18 тема: Сети

Что происходит после ввода адреса сайта в браузере и нажатия Enter?



Сначала браузер разбирает адрес и через DNS узнаёт IP сайта. Потом открывает TCP-соединение, а для HTTPS ещё договаривается о шифровании (TLS) и проверяет сертификат. Дальше отправляет HTTP-запрос, получает ответ, догружает картинки, стили и скрипты и рисует страницу. По дороге запрос может пройти через CDN, балансировщик и прокси.



Вопрос № 19 тема: Базы данных

Какие подходы к бэкапам баз данных вы знаете?

Есть логические копии через `pg_dump`: их легко перенести на другую версию, но восстановят они базу только на момент снятия. Есть физические — через `pg_basebackup`, pgBackRest или WAL-G. Вместе с архивом WAL они позволяют вернуть базу на любую секунду (PITR). Хранят копии по правилу 3-2-1. И главное: бэкап рабочий, только если из него регулярно пробуют восстановиться.

Вопрос № 20 тема: Kubernetes



Что происходит после kubectl apply?

`kubectl` отправляет манифест в `kube-apiserver`. Тот проверяет, кто ты и есть ли у тебя права, прогоняет объект через проверки (admission) и сохраняет в etcd. Дальше всё идёт само и асинхронно: контроллер Deployment создаёт ReplicaSet, тот — поды, планировщик выбирает им ноды, kubelet скачивает образ и запускает контейнеры. Так что успешный `apply` значит только «описание принято», а не «всё уже работает».

Отметь, на что уже отвечаешь уверенно

Галочки можно ставить прямо в этом PDF: нажми на квадратик и сохрани файл.
К остальным вопросам вернись через пару дней.

- [ ] 01 Liveness и Readiness
- [ ] 02 Load Average
- [ ] 03 requests и limits
- [ ] 04 Dockerfile и образы
- [ ] 05 variable precedence
- [ ] 06 Компоненты Kubernetes
- [ ] 07 Deployment, StatefulSet, DaemonSet
- [ ] 08 CMD и ENTRYPOINT
- [ ] 09 Изоляция контейнера
- [ ] 10 Multi-stage сборка
- [ ] 11 df и du
- [ ] 12 Пайплайн для сервиса
- [ ] 13 Медленная база
- [ ] 14 Типы Service
- [ ] 15 Ingress
- [ ] 16 Контейнер и ВМ
- [ ] 17 No space left on device
- [ ] 18 Адрес в браузере
- [ ] 19 Бэкапы баз
- [ ] 20 kubectl apply